

END-TO-END ENCRYPTED GIT SERVICES

Ya-Nan Li

The University of Sydney
Sydney, Australia

Yaqing Song

UESTC
Chengdu, China

Qiang Tang

The University of Sydney
Sydney, Australia

Moti Yung

Google & Columbia University
New York, United States

CCS 2025

1. GIT SERVICE

Introducing the Git service and privacy concerns

2. GIT CANNOT OFFER E2E SECURE

Security, Overhead and Platform Compatibility

3. RESULTS & CONTRIBUTIONS

Formalizing critical security properties: Confidentiality & Repository unforgeability; Balancing overhead & security

4. CONSTRUCTION & EXPERIMENTATION

2 constructions: SGitLine & SGitChar; Conducting experiments on 6 public repos,

WHAT IS GIT?

Git: Distributed Version Control System

1. Tracks file change records and version rollback as repository, provides branch management capabilities
2. Supports multi-person collaborative development and synchronize to Git

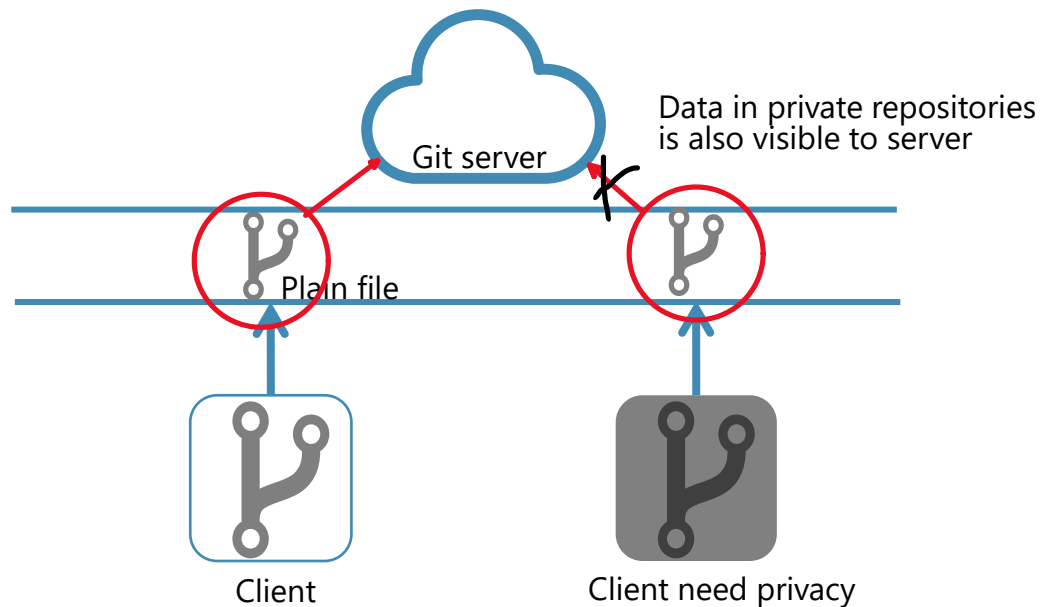


Git is indispensable in the IT industry

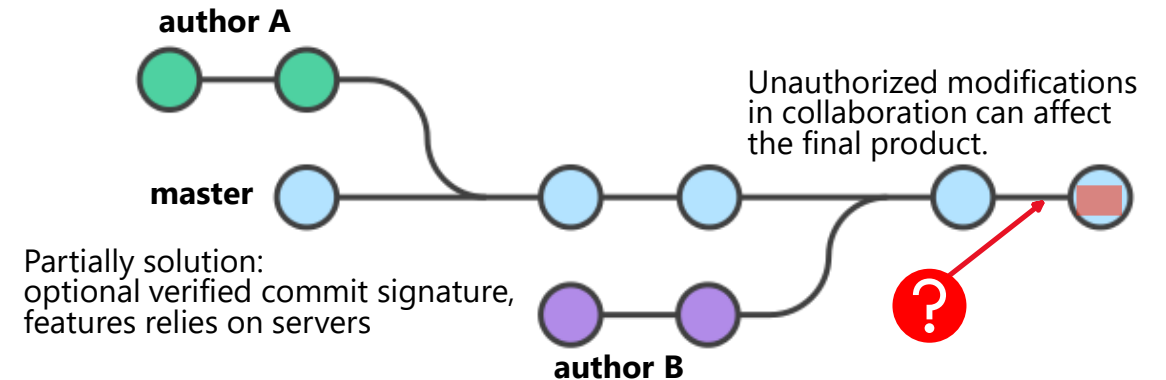
THE RISING DEMAND FOR E2E SECURITY

Why Existing Git Platforms' Security Mechanisms Are No Longer Sufficient?

1. Data is visible to the server.



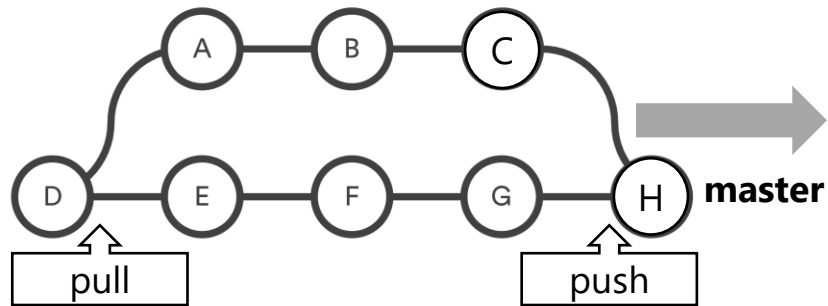
2. Access control relies on the server.



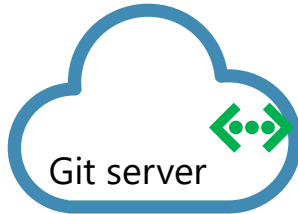
E2E encryption is needed to ensure

E2E SECURE GIT SERVICE IS NOT YET AVAILABLE.

I. E2E Encrypted Cloud Storage Can not Replace E2E Git Security Directly



1. Functional Mismatch
2. Security Inadequacy
3. Efficiency Issues



1. push/pull require cloud to **compute the mini missing parts**
2. Data is updating each time, greater need for access control
3. Save full edit chain lead to strong computation, communication and storage costs;

no Branch merging

Static data

Only final version



Git's unique workflow (dynamic updates, collaborative editing) **requires a special design**

E2E SECURE GIT SERVICE IS NOT YET AVAILABLE.

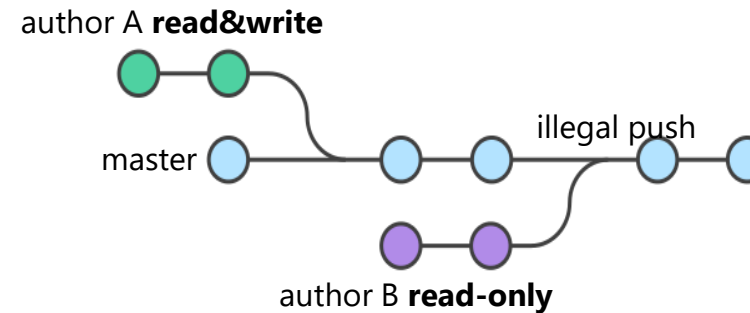
- I. E2E Encrypted Cloud Storage Can not Replace E2E Git Security Directly
- II. **Security risks in existing ad hoc secure Git service designs**

1. Git-crypt & Gringotts: deterministic encryption, weak protection, vulnerable to injection attacks



Mal injection leading to errors(data key) being used later by collaborators .

2. git-secret & Keybase: users choose which file to protect. Entire repository lacks protection.



3. Unforgeability: read-write access separation. No party can assume the rights of any user.

On an untrusted server:

1. Not Confidentiality
2. Failed Integrity
3. Lack of Unforgeability

Need to design **a formal analyzed** E2E encrypted Git service.

E2E SECURE GIT SERVICE IS NOT YET AVAILABLE.

- I. E2E Encrypted Cloud Storage Can not Replace E2E Git Security Directly
- II. Security risks in existing ad hoc secure Git service designs
- III. **Overhead Challenges in Existing Solutions**

Table 1: Comparison with the state-of-the-art “encrypted” Git services.

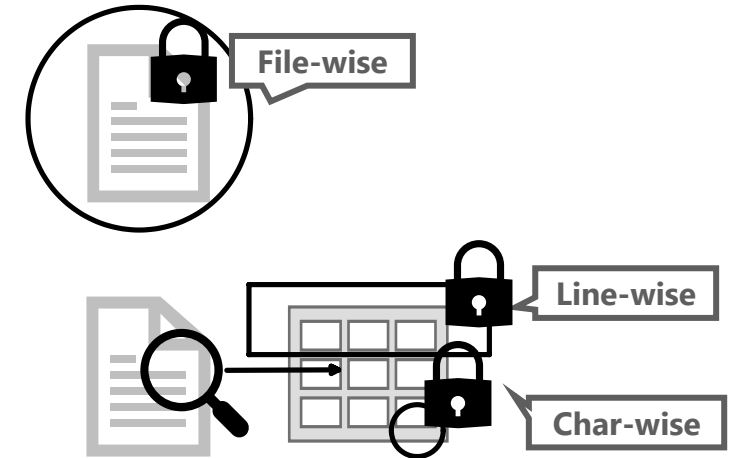
Schemes	Confidentiality	Integrity#	Unforgeability	Storage increase per version†	Client enc cost per update‡	Comm cost per update§	Compatibility**
Git-crypt* [4]	✗	✗	✗	$n_f L$	$n_f L$	$n_f L$	✓
Gringotts* [37]	✗	✗	✗	$n_f \ell_1$	$n_f L$	$n_f \ell_1$	✗
Git-secret [34]	✓*	✗	✗	$n_f L$	$n_f L$	$n_f L$	✓
Git-re-gcrypt [8]	✓	?	?	$n_f L$	$n_f L$	$n_f L$	✗
Disac [38]	✓	?	?	$n_f L$	$n_f L$	$n_f L$	✓
Keybase-Git [9]	✓	?	?	$n_f L$	$n_f L$	$n_f L$	✗
Trivial-enc-sign	✓	✓	✓	R	R	R	✓
Our SGitLine	✓**	✓	✓	$n_f \ell_1$	$n_f \ell_1$	$n_f \ell_1$	✓
Our SGitChar	✓	✓	✓	$n_f \ell_2$	$n_f \ell_2$	$n_f \ell_2$	✓

n_f 表示每个存储库版本中更改文件的数量。

L, R : 分别表示文件和存储库的（平均）大小。

$L1, L2$: 表示每次文件更新时，平均每行更改和每个字符更改的大小。

File-wise, line-wise or char-wise



Goal: Overhead proportional only to actual edits.

E2E SECURE GIT SERVICE IS NOT YET AVAILABLE.

- I. E2E Encrypted Cloud Storage Can not Replace E2E Git Security Directly
- II. Security risks in existing ad hoc secure Git service designs
- III. Overhead Challenges in Existing Solutions
- IV. Platform Compatibility: Working with Existing infrastructure**

Table 1: Comparison with the state-of-the-art “encrypted” Git services.

Schemes	Confidentiality	Integrity#	Unforgeability	Storage increase per version†	Client enc cost per update‡	Comm cost per update§	Compatibility**
Git-crypt* [4]	✗	✗	✗	$n_f L$	$n_f L$	$n_f L$	✓
Gringotts* [37]	✗	✗	✗	$n_f \ell_1$	$n_f L$	$n_f \ell_1$	✗
Git-secret [34]	✓*	✗	✗	$n_f L$	$n_f L$	$n_f L$	✓
Git-re-gcrypt [8]	✓	?	?	$n_f L$	$n_f L$	$n_f L$	✗
Disac [38]	✓	?	?	$n_f L$	$n_f L$	$n_f L$	✓
Keybase-Git [9]	✓	?	?	$n_f L$	$n_f L$	$n_f L$	✗
Trivial-enc-sign	✓	✓	✓	R	R	R	✓
Our SGitLine	✓**	✓	✓	$n_f \ell_1$	$n_f \ell_1$	$n_f \ell_1$	✓
Our SGitChar	✓	✓	✓	$n_f \ell_2$	$n_f \ell_2$	$n_f \ell_2$	✓

Employ E2E encrypted services by simply installing a new secure Git client

Use current Git-provided service operations

E2E SECURE GIT SERVICE 'CHALLENGE

- Identify and formalize critical security properties of an E2E encrypted Git service
- Give provably secure constructions that are both with **minimal overhead** and **platform-compatible** with existing Git servers.

RESULT

- I. Identify and formalize critical security properties of an E2E encrypted Git service
- II. Give provably secure constructions that are both with minimal overhead and platform-compatible with existing Git servers.

Through cryptographic means, an ideal access control similar to a trusted server, achieving:

1. **Confidentiality:** Party without the permission to access confidential data cannot obtain content and update details;
2. **Repository unforgeability:** Party without write permission cannot forge or impersonate the identity of a legitimate user to write data.

Users trust cryptographic methods



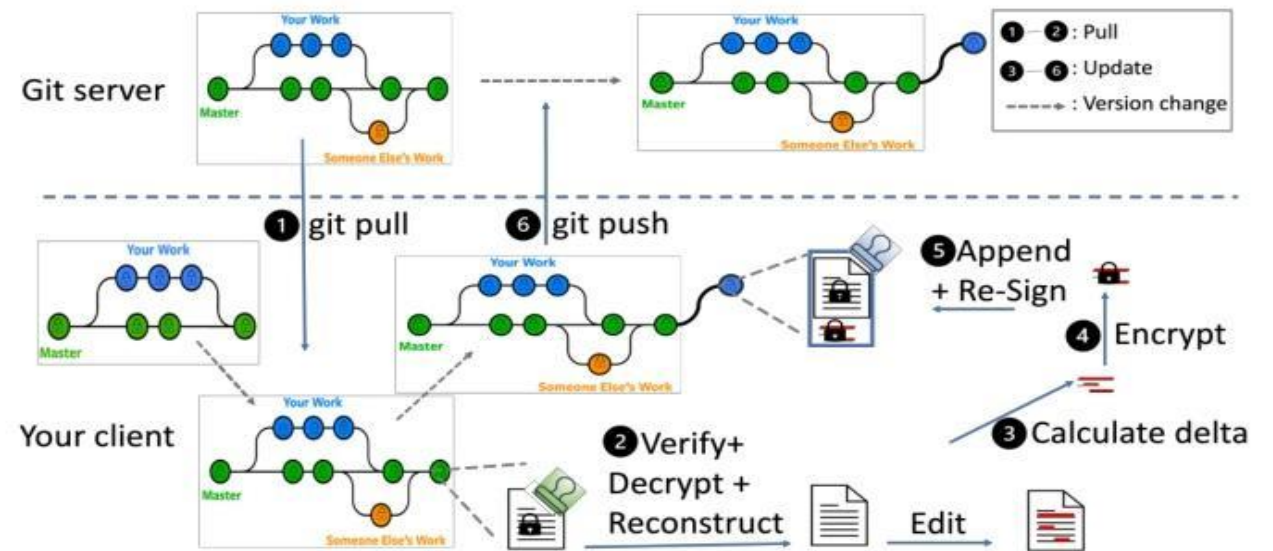
Client

Update details :Operation type (insert/delete),
Modification position (line number, character position),
Modification length

- I. Identify and formalize critical security properties of an E2E encrypted Git service
 - II. Give provably secure constructions that are both with minimal overhead and platform-compatible with existing Git servers.
-
1. Dilemma (Overhead vs. Security):
 - Encrypting **entire files** before pushing disables Git's diff calculation, increasing overhead.
 - **Line-by-line** encryption exposes specific update positions.
 2. Need: - Leverage Git's **diff** functionality
 - No changes to Git platform's overall design

CONTRIBUTIONS

1. Presenting formal syntax and security models for E2E encrypted Git service, data confidentiality and repository unforgeability.
2. 2 constructions: **SGitLine** (line-wise) and **SGitChar** (char-level, update details-hiding), both compatible with existing Git platforms
3. Implementation. Conducting experiments on 6 GitHub repos, and showing overhead advantages.



The main workflow of SGit

SYNTAX & SECURITY MODELS

$$\Pi = (\Pi_{\text{reg}}, \Pi_{\text{auth}}, \Pi_{\text{init}}, \Pi_{\text{update}}, \Pi_{\text{pull}}, \Pi_{\text{shareI}}, \Pi_{\text{shareII}})$$

$$\Pi_{\text{reg}} \langle \text{uid}; st_S \rangle \rightarrow \langle (cred, km); st'_S \rangle$$

$$\Pi_{\text{auth}} \langle (uid, km); st_S \rangle \rightarrow \langle st_U; st'_S \rangle$$

$$\Pi_{\text{init}} \langle (st_U, km, rid, f^{pt}); st_S \rangle \rightarrow \langle repo_{new}; st'_S \rangle$$

$$\Pi_{\text{update}} \langle (st_U, km, rid, repo_{old}, f^{pt}_{new}); st_S \rangle \rightarrow \langle repo_{new}; st'_S \rangle$$

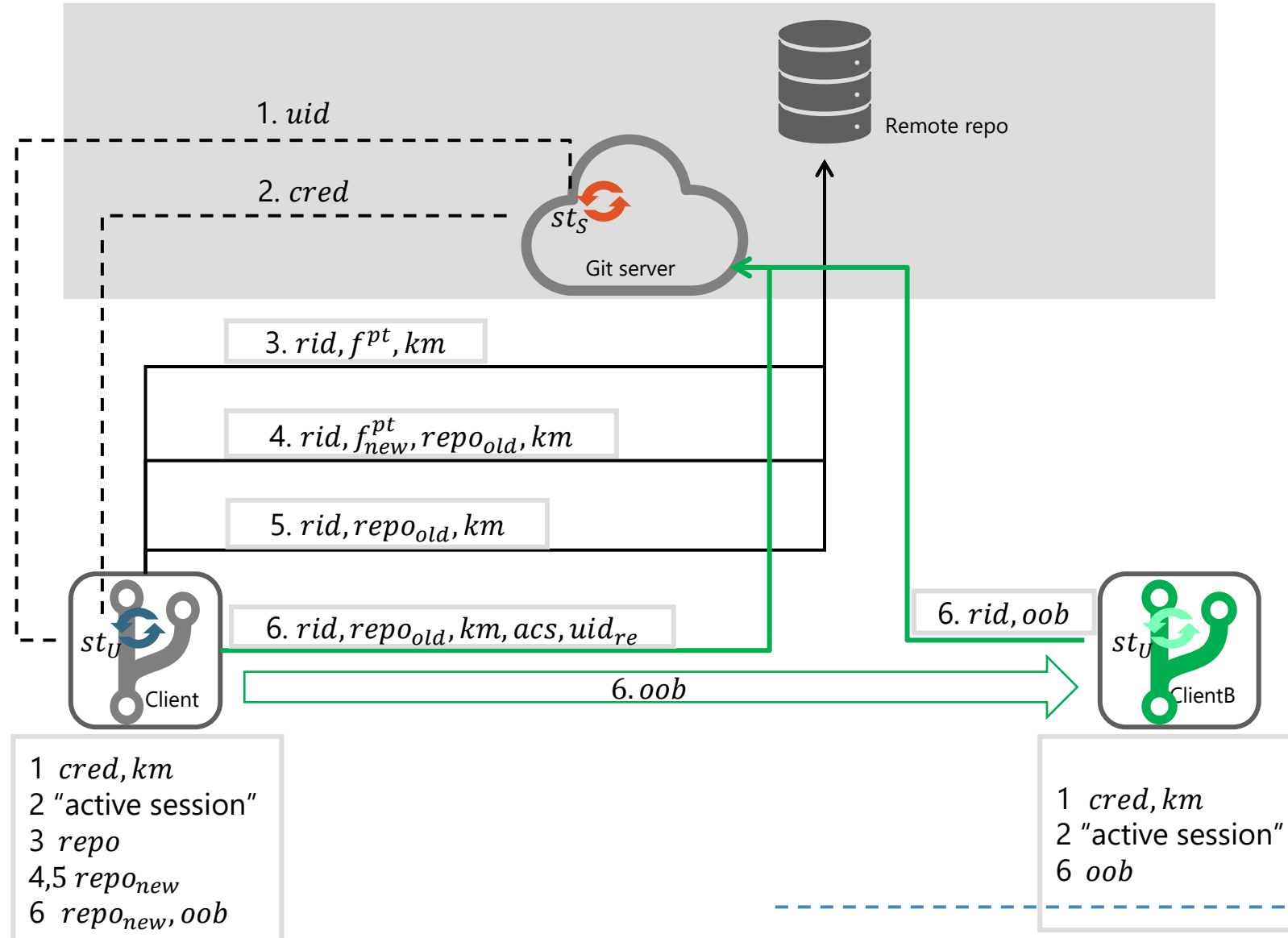
$$\Pi_{\text{pull}} \langle (st_U, km, rid, repo_{old}); st_S \rangle \rightarrow \langle repo_{new}; st'_S \rangle$$

$$\Pi_{\text{shareI}} \langle (st_U, km, rid, acs, repo_{old}, uid_{re}); st_S \rangle \rightarrow \langle (repo_{new}, oob); st'_S \rangle$$

$$\Pi_{\text{shareII}} \langle (st_U, rid, oob); st_S \rangle \rightarrow \langle st'_U; st'_S \rangle$$

Security

1. setup: PKI, OOB.
2. Data confidentiality
3. Repository unforgeability



$$\begin{aligned} \Pi &= (\Pi_{\text{reg}}, \Pi_{\text{auth}}, \Pi_{\text{init}}, \Pi_{\text{update}}, \Pi_{\text{pull}}, \Pi_{\text{shareI}}, \Pi_{\text{shareII}}) \\ \Pi_{\text{reg}} \langle \text{uid}; st_S \rangle &\rightarrow \langle (\text{cred}, km); st'_S \rangle \\ \Pi_{\text{auth}} \langle (\text{uid}, km); st_S \rangle &\rightarrow \langle st_U; st'_S \rangle \\ \Pi_{\text{init}} \langle (st_U, km, rid, f^{pt}); st_S \rangle &\rightarrow \langle \text{repo}_{\text{new}}; st'_S \rangle \\ \Pi_{\text{update}} \langle (st_U, km, rid, \text{repo}_{\text{old}}, f_{\text{new}}^{pt}); st_S \rangle &\rightarrow \langle \text{repo}_{\text{new}}; st'_S \rangle \\ \Pi_{\text{pull}} \langle (st_U, km, rid, \text{repo}_{\text{old}}); st_S \rangle &\rightarrow \langle \text{repo}_{\text{new}}; st'_S \rangle \\ \Pi_{\text{shareI}} \langle (st_U, km, rid, \text{acs}, \text{repo}_{\text{old}}, \text{uid}_{\text{re}}); st_S \rangle &\rightarrow \langle (\text{repo}_{\text{new}}, \text{oob}); st'_S \rangle \\ \Pi_{\text{shareII}} \langle (st_U, rid, \text{oob}); st_S \rangle &\rightarrow \langle st'_U; st'_S \rangle \end{aligned}$$

$Km = ((sk_s, pk_s), (sk_e, pk_e), mk)$
Signature, Encryption, Symmetric master key

$k = KDF(mk, rid)$ Hybrid encryption

$\text{Repo} = (f^{ct}, f_{\text{acs}}, f_{\text{tag}})$
 $f^{ct} = \{ct_1, ct_2, \dots, ct_n\}$
 $f_{\text{tag}} = (\text{uid}, \sigma) : rh = \text{MerkleDAG}(f^{ct} \parallel f_{\text{acs}}),$
 $h = \text{Hash}(rid \parallel uid \parallel rh),$
 $\sigma = \text{Sign}(sk_s, h)$
 $f_{\text{acs}} = \{\text{Read}_{\text{acs}}, \text{Write}_{\text{acs}}, \sigma_{\text{acs}}\};$
 $\sigma_{\text{acs}} = \text{Sign}(sk_s, \text{Read}_{\text{acs}} \parallel \text{Write}_{\text{acs}})$

1. Initialization (reg, auth, init)

2. Update:

ComDiff_{line} and encrypt using Hybrid encryption; Calculate the MerkleDAG hash of the entire ciphertext repository (for integrity verification), and sign with sk_s .

3. Pull: Decrypt line by line

4. Share: $K_{\text{share}} \leftarrow \text{encrypts repo key } k \text{ with } pk_{e(\text{receiver})};$
Update f_{acs}

$$\{O\}_z \leftarrow \text{ComDiff}_{\text{line}}(f, f')$$

$$\text{for } i \in [1, z]$$

$$ct_i \leftarrow \text{Enc}(k, O_i.m), O'_i = (O_i.op, O_i.idx, ct_i)$$

$$ct'_f \leftarrow O'_z(\dots(O'_1(ct_f))), [\{O\}_z \leftarrow \text{ComDiff}_{\text{char}}(f, f')]$$

$$ct_o \leftarrow \text{Enc}(k, \{O\}_z), ct'_f \leftarrow (ct_f, ct_o)$$

Weak data confidentiality: cannot hide updated position & length

Storage: correlated with the number of lines.

- $ComDiff_{char}$, update operations are consolidated into a "difference set" D .
 $D = \{ (i, type, ci'_{ct}) \mid i \in Index, type \in \{modify, add, delete\} \}$
- **Data confidentiality:** conceals updated content, location, and operation type.
- **Efficiency:** correlated with the number of characters.
- **Defect:** pulling to a new local repository requires decrypting all historical differences – may occurs when sharing out.

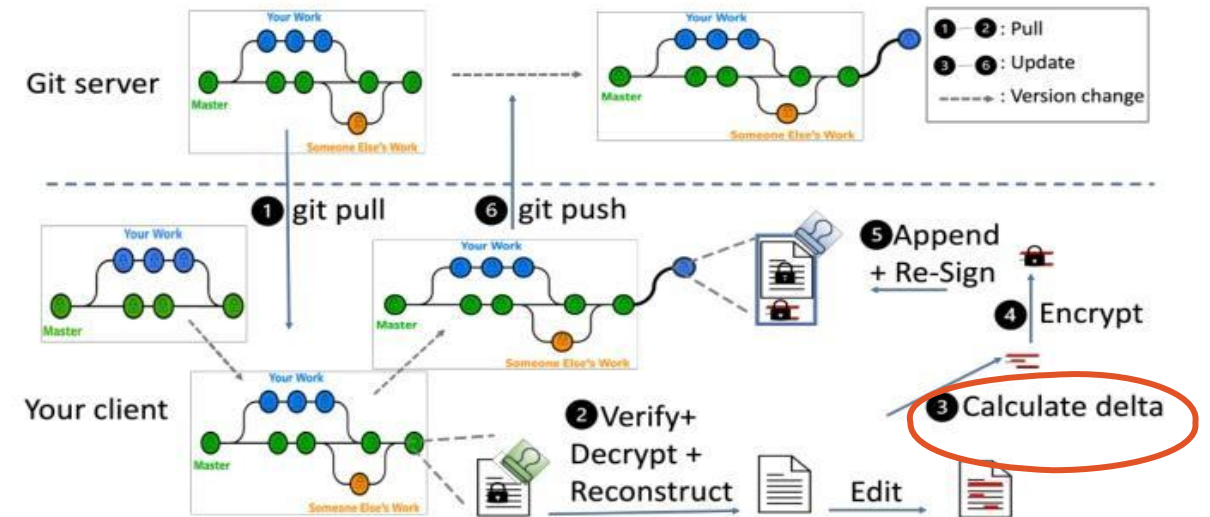
$$\{O\}_z \leftarrow ComDiff_{line}(f, f')$$

$$\text{for } i \in [1, z]$$

$$ct_i \leftarrow Enc(k, O_i.m), O'_i = (O_i.op.O_i.idx, ct_i)$$

$$ct'_f \leftarrow O'_z(\dots(O'_1(ct_f)))$$

$$ct_o \leftarrow Enc(k, \{O\}_z), ct'_f \leftarrow (ct_f, ct_o)$$



IMPLEMENTATION & COMPARISON OF OVERHEAD

Python, pycryptodome library,
AES-CTR as enc
ECDSA as sign
SHA-256 as hash
HKDF-SHA-256 as key
derivation

Table 2: The communication costs of each operation on six repositories using different schemes.

Repo.	Initialization (KB)					One Version Update/Recovery (KB)				
	Git	SGitChar	SGitLine	Git-crypt	Trivial-enc-sign	Git	SGitChar	SGitLine	Git-crypt	Trivial-enc-sign
awesome [3]	0.54	1.06	1.36	1.06	1.06	0.33	0.48	0.44	38.17	0.21 MB
FPB [25]	0.94	1.58	1.99	1.58	1.58	0.41	0.69	0.58	19.62	0.68 MB
bootstrap [10]	253.42	400.50	471.57	400.50	400.50	0.80	4.51	3.31	122.78	2.52 MB
react [33]	620.61	984.00	984.00	984.00	984.00	1.92	8.77	10.20	49.81	23.82 MB
FCC [26]	1.04	1.69	2.30	1.69	1.70	2.39	11.49	11.98	120.61	59.57 MB
DocRepo	445.93	729.38	847.20	719.89	729.45	2.23	10.41	11.01	74.50	0.83 MB

Table 3: The computation overhead of updating six repositories under different schemes.

Repository	SGitChar (s)			SGitLine (s)			Git-crypt (s)	Trivial-enc-sign(s)
	Compare	Encrypt	Total	Compare	Enc+update	Total		
awesome [3]	0.0003	0.0001	0.0004	0.0277	0.0001	0.0278	0.0002	0.0008
FPB [25]	0.0003	0.0001	0.0004	0.0275	0.0001	0.0276	0.0001	0.0045
bootstrap [10]	0.1004	0.0001	0.1005	0.0287	0.0010	0.0297	0.0006	0.0229
react [33]	0.0888	0.0001	0.0889	0.0376	0.0010	0.0386	0.0003	0.1235
FCC [26]	0.0683	0.0002	0.0685	0.0340	0.0009	0.0349	0.0008	0.6045
DocRepo	0.3337	0.0002	0.3339	0.0336	0.0008	0.0344	0.0005	0.0033

Initialize	communication(KB)
update / pull	communication(KB)
	computation- diff (s)
update / pull	computation- encrypt (s)
	computation- all (s)
Pull(recover)	computation(s)
Multi-version store	storage (MB)

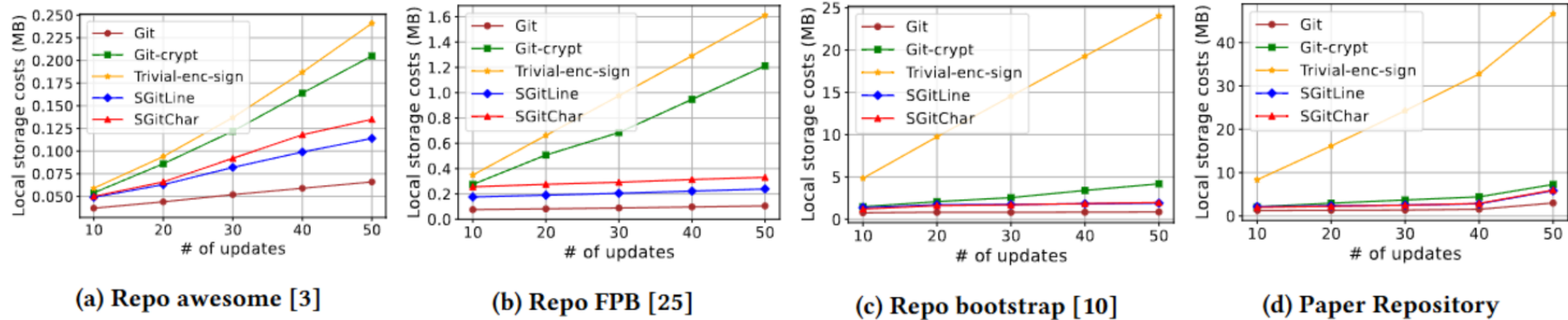


Figure 9: The costs of storing the repositories using different schemes.

IMPLEMENTATION & COMPARISON OF OVERHEAD

awesome	Initialize communication (KB)	Update communication (KB)	Update time (s)	Pull time (s)
SGitChar	1.06	0.48	0.0004	0.0001
SGitLine	1.36	0.44	0.0278	0.0048
Git-crypt	1.06	38.17	0.0002	0.0001
Trivial- enc-sign	1.06	210.0	0.0008	0.0008

•SGitChar: Excellent Initialize / pull performance, low update communication cost, overall balanced;

•SGitLine: Update communication cost is better than SGitChar, but update / pull calculation time is higher;

•Git-crypt: Update calculation time is the lowest, but update communication cost is extremely high;

•Trivial-enc-sign: The various costs (especially communication) are the highest, and the performance is the worst.

1. **Conclusion:** First formal, systematic study on E2E encrypted Git services; formalizes core security properties (confidentiality, unforgeability) , proposes SGitLine/SGitChar
2. Open Problems - **Security:** Extend models to adaptive corruption; strengthen metadata security (hide access/edit patterns)
3. Open Problems - **Functionality:** Support flexible crypto group management, key rotation, revocation, accountability; integrate with web-based Git interfaces
4. **Future Directions:** Explore post-compromise security, Key rotation, Group Management...